

```

    # Keep taking steps until the walk reaches the desired length.
❶ while len(self.x_values) < self.num_points:

    # Decide which direction to go, and how far to go.
❷ x_direction = choice([1, -1])
    x_distance = choice([0, 1, 2, 3, 4])
❸ x_step = x_direction * x_distance

    y_direction = choice([1, -1])
    y_distance = choice([0, 1, 2, 3, 4])
❹ y_step = y_direction * y_distance

    # Reject moves that go nowhere.
❺ if x_step == 0 and y_step == 0:
        continue

    # Calculate the new position.
❻ x = self.x_values[-1] + x_step
    y = self.y_values[-1] + y_step

    self.x_values.append(x)
    self.y_values.append(y)

```

We first set up a loop that runs until the walk is filled with the correct number of points ❶. The main part of `fill_walk()` tells Python how to simulate four random decisions: Will the walk go right or left? How far will it go in that direction? Will it go up or down? How far will it go in that direction?

We use `choice([1, -1])` to choose a value for `x_direction`, which returns either 1 for movement to the right or -1 for movement to the left ❷. Next, `choice([0, 1, 2, 3, 4])` randomly selects a distance to move in that direction. We assign this value to `x_distance`. The inclusion of a 0 allows for the possibility of steps that have movement along only one axis.

We determine the length of each step in the *x*- and *y*-directions by multiplying the direction of movement by the distance chosen ❸ ❹. A positive result for `x_step` means move to the right, a negative result means move to the left, and 0 means move vertically. A positive result for `y_step` means move up, negative means move down, and 0 means move horizontally. If the values of both `x_step` and `y_step` are 0, the walk doesn't go anywhere; when this happens, we continue the loop ❺.

To get the next *x*-value for the walk, we add the value in `x_step` to the last value stored in `x_values` ❻ and do the same for the *y*-values. When we have the new point's coordinates, we append them to `x_values` and `y_values`.

Plotting the Random Walk

Here's the code to plot all the points in the walk:

```

rw_visual.py import matplotlib.pyplot as plt

from random_walk import RandomWalk

# Make a random walk.

```